

```
"""
FastAPI wrapper for SimpleRag.
Drop this file into RAG/SimpleRag/
(alongside app.py, ingest.py) and run:
```

```
    uvicorn api:app --reload --port 8000
```

```
Then open http://127.0.0.1:8000/docs for
interactive Swagger UI.
"""
```

```
import os
from contextlib import asynccontextmanager
from typing import List

from fastapi import FastAPI, HTTPException
from pydantic import BaseModel

from loaders.pdf_loader import load_pdf
from splitter.text_splitter import
split_documents
from embeddings.embedding_model import
get_embedding_model
from vector_store.vector_db import
load_vector_db, create_vector_db
from retriever.retriever import
get_retriever
from llm.llm import get_llm
from prompts.rag_prompt import RAG_PROMPT
```

```

DATA_FOLDER = "data"

# Holds the loaded models/DB so they're
# built once at startup, not per-request
state = {}

@asynccontextmanager
async def lifespan(app: FastAPI):
    print("Loading embedding model...")
    state["embedding_model"] =
get_embedding_model()

    print("Loading vector DB...")
    state["vector_db"] =
load_vector_db(state["embedding_model"])
    state["retriever"] =
get_retriever(state["vector_db"])

    print("Loading LLM...")
    state["llm"] = get_llm()

    print("RAG API ready.")
    yield
    state.clear()

app = FastAPI(title="Simple RAG API",
version="1.0.0", lifespan=lifespan)

# -----
# Schemas

```

```
# -----  
-----  
  
class QuestionRequest(BaseModel):  
    question: str  
  
class AnswerResponse(BaseModel):  
    question: str  
    answer: str  
    sources: List[str]  
  
class IngestResponse(BaseModel):  
    pages_loaded: int  
    chunks_created: int  
    message: str  
  
# -----  
-----  
# Routes  
# -----  
-----  
  
@app.get("/health")  
def health():  
    return {"status": "ok"}  
  
@app.post("/ask",  
response_model=AnswerResponse)  
def ask(request: QuestionRequest):  
    if not request.question.strip():
```

```

        raise
    HTTPException(status_code=400,
    detail="Question cannot be empty")

    retriever = state.get("retriever")
    llm = state.get("llm")
    if retriever is None or llm is None:
        raise
    HTTPException(status_code=503, detail="RAG
    system not initialized")

    # Retrieve top-3 relevant chunks
    docs =
    retriever.invoke(request.question)
    context = "\n\n".join(doc.page_content
    for doc in docs)

    # Build prompt and call LLM
    prompt =
    RAG_PROMPT.format(context=context,
    question=request.question)
    response = llm.invoke(prompt)

    sources =
    sorted({doc.metadata.get("source",
    "unknown") for doc in docs})

    return AnswerResponse(
        question=request.question,
        answer=response.content,
        sources=sources,
    )

```

```

@app.post("/ingest",
response_model=IngestResponse)
def ingest():
    """
        Rebuilds the Chroma vector DB from all
        PDFs in the data/ folder.
        Equivalent to running ingest.py, but
        callable over HTTP.
    """
    if not os.path.isdir(DATA_FOLDER):
        raise
    HTTPException(status_code=404,
    detail=f"'{DATA_FOLDER}' folder not found")

    pdf_files = [f for f in
os.listdir(DATA_FOLDER) if
f.endswith(".pdf")]
    if not pdf_files:
        raise
    HTTPException(status_code=404, detail="No
    PDF files found in data folder")

    all_documents = []
    for pdf in pdf_files:
        docs =
load_pdf(os.path.join(DATA_FOLDER, pdf))
        all_documents.extend(docs)

    chunks = split_documents(all_documents)

    create_vector_db(chunks,
state["embedding_model"])

    # Point the live retriever at the

```

```
freshly rebuilt DB
    state["vector_db"] =
load_vector_db(state["embedding_model"])
    state["retriever"] =
get_retriever(state["vector_db"])

    return IngestResponse(
        pages_loaded=len(all_documents),
        chunks_created=len(chunks),
        message="Vector DB rebuilt
successfully",
    )
```